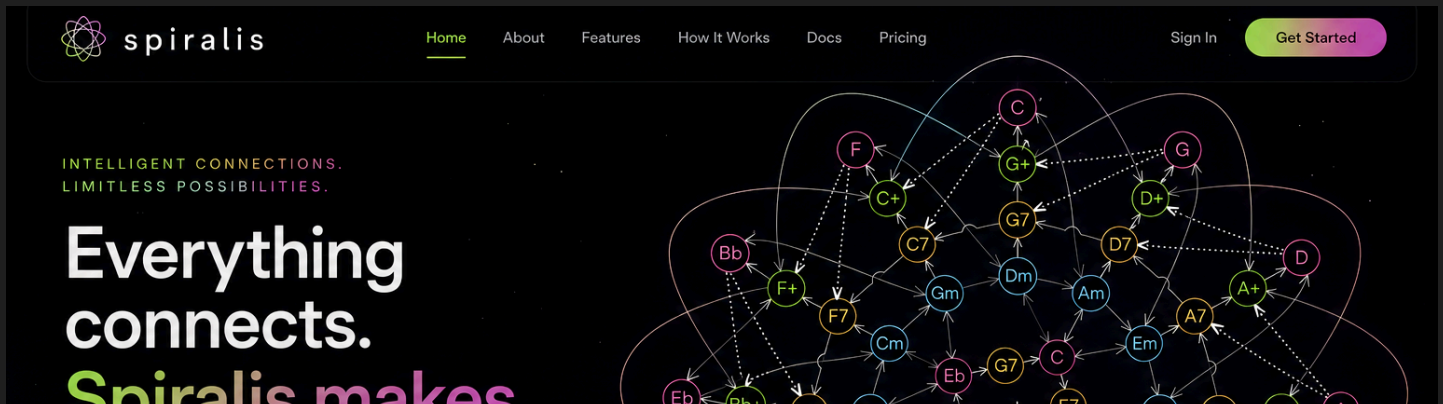




Project Journal

Timing **Sep 1, 2026** to **Dec 12, 2026**

Owner **Erik Siemsen**

Week 1 GOALS/Action Items

Environment, research, and outlining

- Decide on a language and scope of project(web app only, iphone app only, both)
- After Language, find any helpful tools for live recording, chord detection, frequency determination
- Outline work to do until MVP week by week
- Create environments, github files, folders, and fun

Language pros and cons

What each language could bring me along with how familiar i feel with them

Language	Confidence	Pros	Cons	Helpful apis	Api's explained
Python	Primary ▾	Essentia's real implementation lives here; numpy/scipy for any custom DSP; fastest to iterate on the theory engine's logic; FastAPI is lightweight and async-friendly for WebSocket streaming	Slower raw execution than C++ if detection ever needs to run at very large scale; GIL limits true parallelism (not a concern for this project's load)	Essentia, FastAPI, NumPy, SciPy, librosa	Essentia: chord/key/beat detection. FastAPI: async web + WebSocket server. NumPy/SciPy: array math, FFT. librosa: backup/alternative audio analysis utilities.
Java	Tertiary ▾	Strong typing, mature ecosystem, decent audio libraries (TarsosDSP)	Not where Essentia lives; more boilerplate; least practiced of the three, adds risk given the timeline	TarsosDSP, javax.sound	TarsosDSP: pitch detection/onset detection toolkit. javax.sound: low-level audio capture/playback.
C++	Secondary ▾	Essentia is originally a C++ library (Python bindings wrap it) — best raw performance for signal processing	Much slower to build the theory engine and iterate in; no built-in web framework, would need extra glue to expose a web API	Essentia (native), FFTW	Essentia native: same detection engine Python calls into, without the binding layer. FFTW: fast Fourier transform library for custom signal processing.

Week 1 Action Items

Scope Decision

Going with a web app over native mobile. Rationale:

- The primary input mechanism real-time microphone sound is smoothly supported inside browsers through Web Audio API, bypassing redundant audio capture setups across different native systems (like iOS AVAudioEngine or Android AudioRecord).

- Semester time limits made me want to prioritize my theory engine and a working website over getting something all the way to the ios app store.
- A web-based version makes demonstrations easier for weekly updates, mid-term MVP showcases, and final poster presentations requiring only a single URL.

Language and Architecture Choice

Python backend paired with TypeScript React UI, handling audio processing server-side rather than in browser

- Backend: FastAPI in Python delivers streaming analysis via WebSockets to enable live feedback
- Detection: Native Python Essentia handles pitch, chord, key, and beat tracking, avoiding WASM setup complications
- Theory Engine: Custom Python logic covers harmonic relationships and scale matching alongside detector output
- Frontend: React with TypeScript captures audio inputs and renders dynamic visual displays via SVG Canvas

MVP Week-by-Week Outline

Target: working demo by the mid-semester MVP deadline

- Weeks 1–2: Repo setup, FastAPI skeleton, React shell, Web Audio API mic capture streaming raw audio to backend over WebSocket: prove the pipe works end-to-end first
- Weeks 3–5: Essentia integration - key detection on a static file first x, then chord detection
- Weeks 5–7: Theory engine - circle of fifths, scale-fit logic, chord progression paths from root. Pure Python, no audio dependency, so it's built and unit-tested in parallel with detection work
- Weeks 7–8: Wire it all together - live audio in, detection out, theory engine enriches the result, pushed back to frontend, basic visualization (polish ill do after MVP)
- Second half of semester: refine detection accuracy, polish circle-of-fifths wheel and guitar-neck visualizations, incorporate peer/professor feedback, prep poster